

DriftConditions Implementation Plan

Summer 2026 Mobile Apps

1. Purpose

This plan covers the design and development of a native mobile application for DriftConditions, bringing the station's generative audio stream to [iOS](#) and [Android](#) devices. The app is listener-focused: it does not replicate the administrative, editorial, or recipe-authoring interfaces of the web platform. Instead, it delivers the core experience — streaming audio, browsing what's playing, favoriting mixes, and submitting audio — in a form native to how people actually listen to ambient and generative content: with their phone in their pocket, screen off, headphones on.

The existing web platform is built in [React](#) with a well-defined API, [JWT](#)-based authentication, and role-based access control. This document details how those existing systems are extended to support a mobile client, and how the app is structured to stay in sync with ongoing platform development.

2. Goals and Scope

2.1. Goals

1. **Deliver the listening experience natively** — stream DriftConditions audio with full background playback, lock screen controls, and headphone button support on both [iOS](#) and [Android](#).
2. **Surface what's playing** — display the current mix's cover image, playlist, and track metadata in a clean, focused UI.
3. **Support intentional listening modes** — fullscreen mode for immersive display; sleep timer for listeners who fall asleep to the stream.
4. **Enable listener engagement** — allow listeners to favorite a mix.
5. **Enable audio submission** — contributors can submit audio clips from the app, authenticated against the existing role-based system.
6. **Honor the existing auth/authz model** — sign-in, session management, and role-based access reuse the current [JWT](#) infrastructure, adapted for a mobile context.
7. **Ship to both app stores** — publish to Google Play and the Apple App Store with a single shared [React Native](#) codebase.

2.2. Why these goals matter

The DriftConditions stream is ambient by nature — the kind of audio people put on to work, sleep, or move through the world. That use pattern is fundamentally mobile. The current web player supports background audio and lock screen controls via the [Media Session API](#), but

mobile browsers throttle background tabs — interrupting playback when the screen locks or the user switches apps. A native app eliminates that constraint. Favoriting gives listeners a lightweight way to engage with the stream. Audio submission matters because contributors are a key part of what keeps the station alive; removing friction from their workflow — including on mobile — supports the health of the library.

2.3. Technology Choice

Three approaches were evaluated:

- **PWA (Progressive Web App):** Zero new infrastructure, installable from a browser. Rejected because Apple restricts background audio on **iOS** — a hard constraint for a streaming app.
- **Capacitor:** Wraps the existing **web frontend** in a native shell with access to native APIs. A reasonable option, but its primary advantage is reusing existing HTML/CSS UI — which doesn't apply here, since the mobile UI is a new, stripped-down interface rather than a port of the admin-heavy web client.
- **React Native:** Builds a native UI using **React** component patterns. The existing frontend is already **React**, so hooks, state management, and API call patterns transfer directly. The **react-native-track-player** library provides purpose-built background audio, lock screen controls, and headphone integration for both platforms. **Selected.**

2.4. Scope

2.4.1. Audio Streaming

- Live stream playback via **react-native-track-player**
- Background audio with OS-level lock screen and headphone controls
- Play/stop controls; auto-reconnect on stream dropout

2.4.2. Playlist and Cover Display

- Current mix hero/cover image
- Playlist of recent and upcoming tracks
- Mix metadata (title, contributors)

2.4.3. Listening Modes

- Fullscreen mode: dark overlay, large cover art, dismissable
- Sleep timer: auto-stop after a user-set duration

2.4.4. Listener Engagement

- Favorite/star a mix

2.4.5. Audio Submission

- Upload an audio file from the device
- Requires authentication as a Contributor or higher
- Reuses the existing [/api/audio](#) upload endpoint

2.4.6. Authentication

- Sign-in and sign-out using the existing auth endpoints
- JWT stored securely on device (adapted from the web platform's HTTP-only cookie model)
- Sliding session renewal matching the web platform's token refresh logic
- Role-based access: submission UI gated on Contributor role

2.4.7. Stretch Goals

- Cast to devices (Chromecast, AirPlay)

2.5. Out of Scope

- Admin interfaces: audio moderation, recipe authoring, user management (audio submission is the sole exception)
- OAuth / social sign-in (covered in the [Auth Upgrade plan](#); can be added once implemented on the platform)
- Any changes to [MixEngine](#), [AdminServer](#), or the existing [web frontend](#)

3. Implementation Approach

3.1. Project Structure

The mobile app lives in a new top-level directory, [MobileApp/](#), within the existing DriftConditions monorepo. This standalone React Native project is initialized with the React Native CLI to avoid complications with background audio and native module access. The client communicates exclusively with the existing [AdminServer](#) API, reusing current endpoints without requiring any new backend routes.

3.2. Key Dependencies

This project relies on several key dependencies to deliver native functionality. Selecting these packages allows us to handle crucial system behaviors like background audio and secure storage. The table below outlines the primary libraries and their respective roles in the mobile application architecture.

Library	Purpose
<code>react-native-track-player</code>	Background audio, lock screen controls, headphone buttons
<code>react-native-async-storage</code>	Secure JWT storage on device
<code>react-navigation</code>	Screen navigation (stack + tab)
<code>react-native-image-picker</code>	Audio file selection for submission
<code>react-native-safe-area-context</code>	Notch/status bar handling across devices

3.3. Authentication Adaptation

Authentication requires adapting the existing JWT model to the mobile environment to maintain security and session logic. Because native apps cannot access HTTP-only cookies, the mobile client stores the JWT in `AsyncStorage` and attaches it as a `Bearer token` on each request. The existing sliding session renewal logic is replicated client-side, ensuring the system reuses all current `/api/auth` endpoints without any changes to the `AdminServer`. Role-based access is enforced client-side by inspecting the role claim in the decoded JWT, mirroring the web platform's approach; the submission screen is only reachable by authenticated `Contributors` or higher.

3.4. Phased Approach

The implementation is structured into four distinct phases, ensuring a controlled path of change and progressive feature delivery. We order the phases to front-load the highest-risk work, specifically audio playback and authentication, guaranteeing core stability before moving to engagement and submission features. Each phase concludes with a clear exit criterion that verifies the delivered functionality.

3.4.1. Phase 1 — Foundation

Establish the project, get audio playing, and wire up authentication. Everything else depends on these two things working reliably. Testing infrastructure is also set up in this phase, since background audio and lock screen behavior must be verified on physical hardware — simulators do not reproduce it reliably.

- Initialize `React Native` project; configure build tooling for `iOS` and `Android`
- Configure `Android` Emulator and `iOS` Simulator for development testing
- Set up `TestFlight` for `iOS` pre-release distribution; configure `Google Play Internal Testing track` for `Android`
- Verify background playback and lock screen controls on physical devices (both platforms) before closing the phase

- Implement sign-in and sign-out screens; **JWT** storage and request middleware
- Integrate **react-native-track-player**; connect to the live stream URL
- Basic play/stop UI
- Navigation shell: tab bar or stack structure for the screens to follow

*Exit criterion: authenticated user can sign in, start the stream, lock a physical **iOS** device and a physical **Android** device, and control playback from the lock screen on both. **TestFlight** and Internal Testing builds are live and installable by the development team.*

3.4.2. Phase 2 — Core Listener Experience

Phase 2 builds the core listener experience, focusing on the screens a listener will spend most of their time using. This phase surfaces the currently playing content, including the playlist and mix metadata. The items below detail the features and user modes that define the primary listening UI.

- Playlist screen: current mix cover image, track list, mix metadata
- Fullscreen mode: dark overlay, large cover art, dismissable
- Sleep timer: user-set duration, auto-stop with fade

Exit criterion: a listener can open the app, see what's playing, go fullscreen, and set a sleep timer — without ever needing to sign in.

3.4.3. Phase 3 — Engagement and Submission

Phase 3 integrates engagement and submission capabilities, which require user authentication and write access. This ensures that we defer work that relies on a stable authentication system until the core architecture is proven stable. The following features enable listeners to favorite content and contributors to upload audio.

- Favorite/star a mix; persist to the existing favorites API
- Audio submission: file picker, upload form, progress feedback, error handling
- Contributor-gated navigation (submission screen hidden or locked for Viewer role)

Exit criterion: a contributor can submit an audio file from their phone; any listener can favorite a mix.

3.4.4. Phase 4 — Polish and Release

Phase 4 prepares the application for submission to the Google Play and Apple App Stores by focusing on final polish and addressing fit-and-finish items. This includes branding consistency, platform-specific testing, and generating all required store assets. The final step is submitting the verified app to both stores for approval.

- Branding: app icon, splash screen, colors consistent with DriftConditions visual identity
- Platform-specific polish: safe areas, permission dialogs, offline/error states

- Testing on physical devices (iPhone and Android)
- App store assets: screenshots, descriptions, metadata
- Submit to Google Play and Apple App Store
- Stretch goal: Cast integration (Chromecast / AirPlay)

Exit criterion: app approved and live on both stores.

4. UI Specifications

4.1. Design Principles

The mobile UI strips away all administrative and editorial tooling, focusing exclusively on the core listener experience. The interface honors the ambient, generative nature of the stream by avoiding unnecessary visual noise. Screens are minimal, navigation is discoverable, and the persistent player ensures the stream never stops when moving between views.

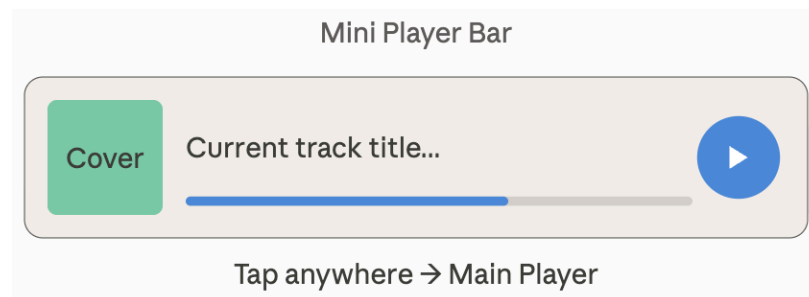
The design echoes familiar patterns from podcast and music streaming apps (Pocket Casts, Apple Music, Spotify) while respecting the unique character of a generative, never-repeating audio source. There are no discrete "tracks" — only an ongoing stream with metadata about what's currently playing.

4.2. Persistent Elements

Two UI elements persist across all screens: the mini player bar and the control bar. These components ensure continuous playback and consistent access to core features regardless of which screen the user is viewing.

4.2.1. Mini Player Bar

The mini player appears at the bottom of all screens except the main player screen. It provides minimal playback information and a quick return path to the full player interface.



Layout:

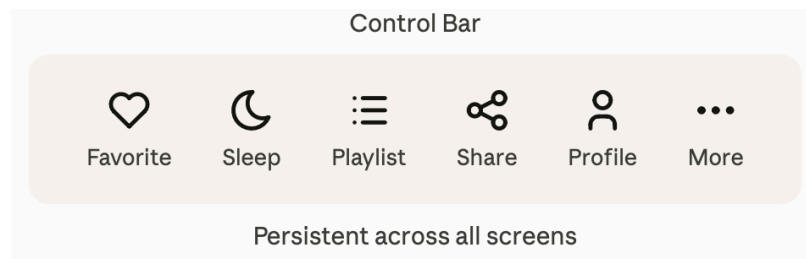
- Small cover art thumbnail (left edge)
- Play/pause button (right edge)
- Tap anywhere on the bar → navigate to main player screen

Behavior:

- Hidden on the main player screen (that screen IS the player)
- Remains visible and interactive on all other screens
- Reflects current playback state (playing/paused)

4.2.2. Control Bar

The control bar appears below the mini player bar (or below the main player on the home screen) and provides access to all primary listener actions. It persists across every screen in the app.

**Layout (left to right):**

1. Favorite (heart icon)
2. Sleep timer (moon icon)
3. Playlist (list icon)
4. Share (share icon)
5. Profile (person icon)
6. More (dot-dot-dot icon)

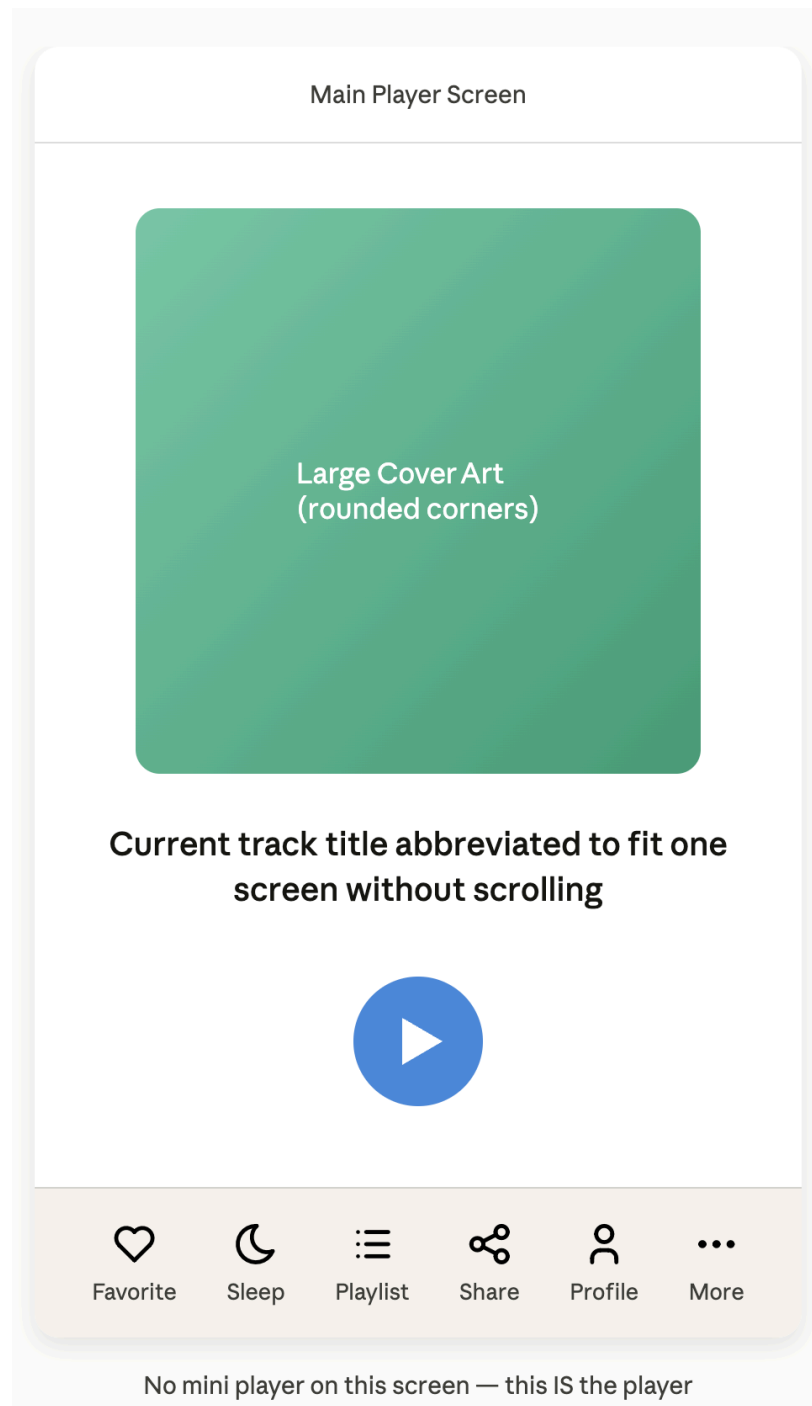
Behavior:

- Always visible at bottom of screen
- Icons update state visually (e.g., filled heart when favorited, moon with countdown when timer active)
- Tapping an icon triggers the associated action or navigates to the associated screen

4.3. Screens

4.3.1. Main Player Screen

The main player is the default landing screen. It displays the currently playing content in a focused, immersive layout.



Layout:

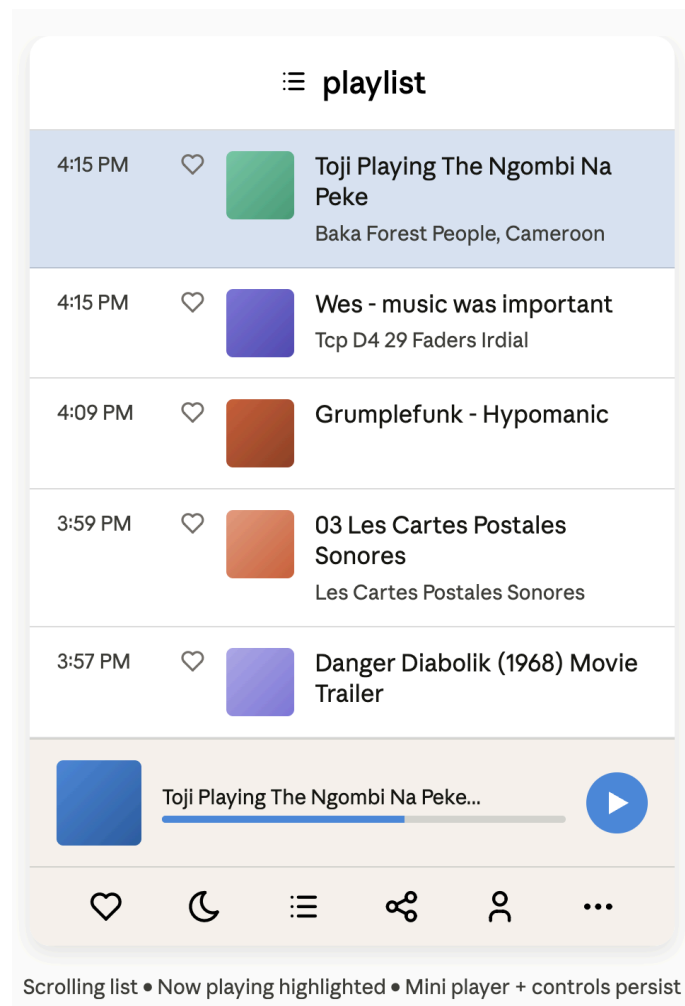
- Large cover art with rounded corners (centered, occupies most of vertical space)
- Track title below cover art (abbreviated to fit one screen without scrolling)
- Large play/pause button below title
- Control bar at bottom

Behavior:

- No mini player bar (this screen replaces it)
- Cover art and title update when the mix changes
- Play/pause button reflects current playback state

4.3.2. Playlist Screen

The playlist screen displays a scrolling list of recent and upcoming playlist entries, providing context for what's playing and what played before.



Layout:

- Scrolling list of entries, each showing: timestamp, title, small cover thumbnail
- "Now playing" indicator on the current entry
- Mini player bar at bottom
- Control bar below mini player

Behavior:

- List scrolls independently; mini player and control bar remain fixed
- Tapping an entry does nothing (entries are informational only; the stream is continuous)

4.3.3. Upload Screen

The upload screen allows contributors to submit audio files directly from their device. This screen is only accessible to users with Contributor role or higher.

Layout:

- File picker button
- Title text input field
- Classification dropdown (single select)
- Tags text input field
- Submit button
- Mini player bar at bottom
- Control bar below mini player

Behavior:

- Screen is hidden or locked for users without Contributor role
- File picker opens device file browser
- Submit button uploads to existing [/api/audio](#) endpoint
- Progress feedback during upload
- Error handling for failed uploads

4.3.4. Profile Screen

The profile screen displays user information and provides access to account actions.

Layout:

- User display name
- User email
- Role badge
- Sign out button

- Mini player bar at bottom
- Control bar below mini player

Behavior:

- If not authenticated, displays sign-in prompt instead
- Sign out clears stored JWT and returns to unauthenticated state

4.3.5. Auth Screen

The auth screen provides the sign-in interface for unauthenticated users.

Layout:

- Email text input field
- Password text input field
- Sign in button
- Mini player bar at bottom (if stream is playing)
- Control bar below mini player

Behavior:

- Successful sign-in stores JWT and navigates to main player or previous screen
- Error handling for invalid credentials
- Stream continues playing during sign-in process

4.3.6. About Screen

The about screen provides project information, credits, and external links. Drawn from the about text on the web app main page.

DriftConditions

An unfolding mystery in a unique journey

About

DriftConditions is an online radio station that captures the warmth of voice and the chill of the unknown in a world that continually assembles and reassembles itself.

This online audio experience weaves a fresh soundscape from community contributions, ambient recordings, and procedural composition.

Credits

Created by Wes Modes



Current track playing...



White logotype on #336699 brand header

Layout:

- Project description text (scrollable)
- Credits section
- External links (project website, source code repository)
- Mini player bar at bottom
- Control bar below mini player

Behavior:

- External links open in device default browser
- Content is read-only

4.4. Modals

Modals appear as overlays above the current screen, dimming the background content. They are dismissed by tapping outside the modal or using an explicit close/cancel action.

4.4.1. Sleep Timer Modal

Triggered by tapping the moon icon in the control bar. Allows the user to set an auto-stop timer.

Layout:

- Modal title: "Sleep Timer"
- Preset buttons: 15 min, 30 min, 45 min, 60 min
- Cancel button

Behavior:

- Tapping a preset sets the timer and closes the modal
- Moon icon in control bar updates to show countdown
- Stream fades out and stops when timer expires
- Cancel button closes modal without setting timer

4.4.2. More Menu Modal

Triggered by tapping the dot-dot-dot icon in the control bar. Provides access to secondary features.

Layout:

- Menu items (vertical list):
 - Upload (visible only to Contributors or higher)
 - About
 - Settings

Behavior:

- Tapping a menu item navigates to the associated screen and closes the modal
- Tapping outside the modal closes it without action

8. Appendices

8.1. Appendix A — Document Style Guide

This section details the standardized formatting applied to the document.

Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)
- **Font Weight:** Normal
- **Font Color:** Black
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.
- **Structural Patterns:** H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

Body Text

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Font Weight:** Normal
- **Font Color:** Black
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and code blocks.

Numbered or Bulleted Lists

Numbered or bulleted lists follow the Body Text standard. They are always single spaced with no before or after spacing between list items.

- **Spacing (Before List):** 0pt
- **Spacing (After List):** 12pt
- **Spacing (Before List Item):** 0pt
- **Spacing (After List Item):** 0pt

Tables

All tables are compact and readable with minimal line breaks and no trailing linefeeds within cells.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Font Weight:** Normal
- **Font Color:** Black
- **Line Spacing:** 1.15
- **Spacing (Before Text):** 0pt
- **Spacing (After Text):** 0pt

Inline Technical References

This style, often referred to as <tt> or console text, is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)
- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

Code Block Syntax Highlighting Style: One Light (Notion variant)

This applies to any multiline code/SQL on its own lines. Use the GoogleDoc CodeBlock building block. A light-background syntax theme derived from the One Light family, originally developed for the Atom editor. Clean, low-contrast, and readable on white or near-white backgrounds.

Background: Light gray (#f7f7f7 or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names), radiused corners.

Typography: 10pt, Monospace, medium weight, single-space line height, no before or after spacing. Token colors are distinct but not garish — optimized for readability over visual drama.

Token colors:

- **Keywords** (`ALTER`, `CREATE`, `ADD`, `NOT`, `NULL`, `ON`, `DELETE`): Medium blue (#0070c1 or

similar)

- **Types** (`INT`, `VARCHAR`, `DATETIME`): Bright blue (`#0099cc` or similar)
- **Numeric literals** (`255`, `32`, `191`): Orange-red (`#e06c00` or similar)
- **Comments** (`-- ...`): Muted pink/mauve (`#c678a0` or similar)
- **Language label** (`SQL`): Small-caps, light gray, non-highlighted

SQL

```
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userID) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```